

A Real-Time Channel-Level Intrusion Detection System Based on Multimodal Learning

Xiaowei Zhao^{ID}, Mingshu He^{ID}, *Member, IEEE*, and Xiaojuan Wang^{ID}, *Member, IEEE*

Abstract—With the proliferation of Internet of Things (IoT) devices, cybersecurity threats are escalating. To protect user privacy and data integrity, a significant portion of IoT traffic is secured using encryption technologies. In addition, certain IoT scenarios demand that intrusion detection systems (IDSs) process traffic in real-time. Thus, encrypted flow detection and real-time detection have emerged as two core challenges for IDS in IoT. To address these challenges, this article proposes RCML-IDS, a real-time channel-level IDS based on multimodal learning. The core novelty of RCML-IDS lies in its real-time, online processing capability, enabled by a time-window-based traffic preprocessing mechanism. In addition, it performs channel-level traffic aggregation and integrates multimodal features, namely raw bytes and packet lengths, to capture rich behavioral patterns from encrypted traffic. Architecturally, two Transformers learn multilevel byte representations from local to global contexts, while an LSTM captures temporal patterns in packet length sequences. To the best of the authors' knowledge, this is the first multimodal IDS capable of real-time online traffic processing. The experimental results demonstrate that the RCML-IDS outperforms existing approaches on public and self-collected datasets. Its lightweight version achieves a per-sample processing time of approximately 20 ms and permits deployment on resource-constrained devices, offering an effective solution for IoT security.

Index Terms—Channel-level traffic aggregation, multimodal learning, real-time intrusion detection.

I. INTRODUCTION

NOWADAYS, the number of connected devices in the Internet of Things (IoT) has significantly increased, and cyber threats have become increasingly severe [1]. According to the DDoS insights report released by Zayo Group, the frequency of IoT attacks surged by nearly 82% between 2023 and 2024 [2]. In response to this phenomenon, experts have

pointed out that attackers are using larger botnets composed of infected IoT devices and artificial intelligence (AI) to significantly increase the scale of attacks [3]. Against this backdrop, intrusion detection systems (IDSs) have become a key technological means for safeguarding IoT security.

However, in the context of IoT, IDSs primarily face two core challenges: encrypted traffic detection and real-time detection requirements.

On the one hand, data transmitted in IoT commonly contains user privacy and sensitive information. To protect the security of such data, encryption technologies are typically employed for secure transmission [4], [5]. This renders traditional port-based and payload-based methods largely ineffective [6]. Although existing research has attempted to detect encrypted traffic using packet length sequence features [27], [28], [29], [30], [31], [32], these approaches lose too much data detail. Relying solely on a single modality makes it difficult to capture complex attack patterns in encrypted traffic, resulting in limited generalization capabilities. Consequently, multimodal learning offers a more effective solution for encrypted traffic detection.

Multimodal learning, as a method capable of processing and understanding information from different modalities, aligns more closely with the multisensory collaborative way in which humans perceive the world [7]. In the field of network intrusion detection, extracting features from multiple modalities of network flows and fitting features with different attributes using various deep learning (DL) models has been proven to be an effective strategy [8], [39], [40], [41], [42], [43], [44]. Therefore, this article adopts two modalities of features: raw bytes and packet lengths. As illustrated in Fig. 1, these can be analogized to image and audio signals in the physical world. Specifically, raw bytes contain the specific content of network traffic, akin to the pixel information in an image, and many works have converted them into grayscale images [33], [34], [35]; packet length sequences reflect the changes in packet sizes over time, similar to the amplitude variations of audio signals at different time points. The fusion of images and audio has been widely applied in areas such as action recognition [9], vehicle classification [10], and fake video detection [11]. Similarly, by fusing raw bytes and packet lengths, we aim to reveal the complex patterns of network flows from different perspectives, thereby enhancing the performance of encrypted traffic intrusion detection in IoT.

On the other hand, in IoT scenarios, especially in vehicular networks and industrial IoT, there is a high demand for

Received 9 April 2025; revised 19 October 2025; accepted 1 January 2026. Date of publication 12 January 2026; date of current version 26 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62402053 and Grant 62227805 and in part by the Fundamental Research Funds for the Central Universities under Grant 2025KYQD17(BUPT). (Corresponding author: Mingshu He.)

Xiaowei Zhao is with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: xwzhao@bupt.edu.cn).

Mingshu He is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with the Key Laboratory of Trustworthy Distributed Computing and Service, Ministry of Education, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: hemingshu@bupt.edu.cn).

Xiaojuan Wang is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: wj2718@bupt.edu.cn).

Digital Object Identifier 10.1109/IJOT.2026.3651905

2327-4662 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on May 18, 2026 at 09:55:35 UTC from IEEE Xplore. Restrictions apply.

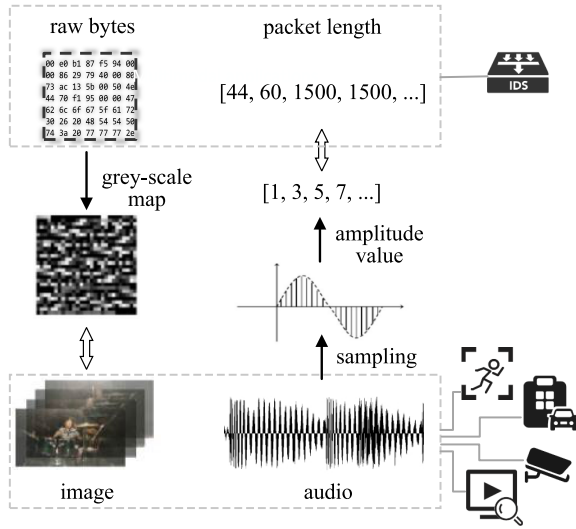


Fig. 1. Analogical relationships between byte/packet-length features in network traffic and physical-world image/audio signals.

real-time IDSs to promptly identify and respond to security threats. If the system experiences delays, it may provide opportunities for attackers to exploit vulnerabilities. Existing research primarily focuses on optimizing model complexity to improve training efficiency [1], [45], [46], [47], but these approaches struggle to handle dynamic and continuous online data streams, making them difficult to deploy in practical applications.

In addition, traditional IDSs commonly employ flow-level analysis, but their aggregation method based on five-tuples overlooks long-term interaction patterns between hosts [12], [39]. For example, in port scanning attacks, attackers probe multiple ports of a target host to discover open ports and services. Such behaviors are more easily exposed at the channel level (based on persistent IP pair connections).

To address the aforementioned challenges of insufficient single-modal feature representation in encrypted traffic and high real-time requirements for IoT traffic detection, this article proposes a real-time channel-level IDS based on multimodal learning, named RCML-IDS.¹ The system supports encrypted traffic detection and possesses real-time capabilities. The main contributions of this article are summarized as follows.

- 1) We design an online traffic preprocessing mechanism based on time-window sampling, which generates observations consistent with those collected in existing online systems, providing reliable data for simulating real-time attack detection. Notably, the detection latency of this preprocessing mechanism is largely determined by the size of the time window (e.g., 1.10-s latency with 1-s time windows), enabling it to flexibly meet the needs of online real-time detection.
- 2) We propose a multimodal learning framework capable of learning two feature modalities: raw bytes and

packet lengths, to enhance detection performance in both encrypted and nonencrypted scenarios. By optimizing a multiloss function, the model can better integrate and balance the information learned from byte- and length-level features. Ablation studies demonstrate that our multimodal design achieves a 14% accuracy improvement over its single-modality variant using exclusively packet length features.

- 3) Unlike traditional methods that operate at the flow level, we aggregate interhost traffic at the channel level. This coarse-grained approach allows us to comprehensively capture richer behavioral patterns. Furthermore, we employ two Transformer structures to achieve multilevel byte feature learning: the packet-level byte Transformer uses self-supervised pretraining to capture intrapacket byte relationships, and experiments have shown that this pretraining strategy can improve detection accuracy by 4%; the channel-level byte Transformer models interpacket correlations within a channel.

The rest of this article is organized as follows. Section II summarizes the related work and analyzes existing methods. Section III describes the preliminaries. Section IV introduces the detailed design of the proposed method. Section V presents the experimental results and performance evaluation. Finally, Section VI concludes this article.

II. RELATED WORKS

Traditional intrusion detection algorithms, such as port-based traffic classification [13] and deep packet inspection [14], are rarely used in encrypted flow detection tasks because they are almost incapable of handling malicious traffic protected by protocols, such as TLS, VPN, and Tor [15]. As a result, in recent years, numerous feature-based IDS have emerged, which typically combine machine learning (ML) and DL algorithms to learn statistical and sequential features. However, the classification performance of single-feature methods is limited. Therefore, some studies have adopted multimodal DL models to address this issue. Table I compares our method with existing works in terms of the model employed, features used, and the support for both encrypted traffic detection and real-time detection.

A. Statistical Features-Based Models

Statistical feature-based detection methods rely on extracting features from bidirectional flows (e.g., total number of packets, packet arrival time intervals, and flow duration) [16], [17], [18], [19], [20], [21], [22], [23], [24]. These statistical features are typically modeled and analyzed using ML algorithms to enable threat identification in encrypted traffic [13]. For example, Apruzzese et al. [19] proposed a random forest (RF)-based IDS using features, such as total transmitted packets, total transmitted bytes, and bytes per second, which demonstrated certain resilience against adversarial attacks. Mohanty et al. [21] combined the predictions from three meta-learners—RF, K -nearest neighbors (KNN), and decision tree (DT)—to develop a meta-learning-based defense mechanism, enhancing the robustness of statistical feature classification.

¹The source code is publicly available at <https://github.com/Akap-vv/RCML-IDS>

TABLE I
COMPARISON OF OUR APPROACH WITH EXISTING WORKS

Modality	Ref	Model	Features used	Encrypted Detection	Real-time Detection
Single modal	[19]	RF	Statistical features	×	×
	[21]	RF-KNN-DT		×	×
	[23]	AdaBoost		×	×
	[24]	C4.5 algorithm		×	×
	[27]	LSTM	Packet length sequences	✓	×
	[28]	GRU		✓	×
	[30]	GNN		✓	×
	[33]	CNN		✓	×
	[36]	Transformer	Byte sequences	✓	×
	[37]	CNN-BiLSTM		✓	×
	[38]	MAE		✓	×
	[1]	IResTAE2A		×	✓
	[45]	RF	Statistical features	×	✓
	[46]	HDC		×	✓
	[47]	Meta-learning		×	✓
Multi modal	[39]	EdgeNet	Bytes, lengths, time intervals	✓	✓
	[40]	MDAE	Multi-level features	×	×
	[41]	CNN-GRU	Fused features	×	×
	[42]	Multi-kernel convolution	Packet length, window size, TCP flags	✓	×
	[43]	GNN	Packet length sequences, statistical features	✓	×
	[44]	CNN-LSTM	Bytes, packet length	✓	×
	Our	Transformer-LSTM	sequences	✓	✓

Moustafa et al. [23] generated new statistical flow features by analyzing the latent characteristics of protocols and employed the AdaBoost algorithm to detect malicious traffic in IoT environments. Zhang et al. [24] established the threat models for various attacks based on statistical flow features and authentication mechanisms to protect IoT systems.

However, these methods rely heavily on feature selection, which requires extensive experience and specialized knowledge. However, another limitation of such approaches is that they necessitate observing all packets within a complete network flow to form the corresponding flow statistics. Therefore, these methods are greatly time-consuming and unsuitable for scenarios with high real-time requirements.

B. Sequence Features-Based Models

Sequence feature-based methods focus more on the temporal information of traffic to explore characteristics, such as trends and periodicity within sequences. Commonly used features include packet length sequences, time-interval sequences, and byte sequences. However, packet timing is susceptible to network conditions, making it unstable [25]. In addition, existing research that uses packet time intervals as features often employs dynamic time warping [26], which incurs significant computational overhead during feature extraction. As a result, packet length and byte sequence features are more widely utilized.

The intuition behind using packet lengths is that attack traffic may exhibit different length distributions and variation patterns compared to normal traffic, which can be leveraged to analyze IoT network traffic and detect potential attacks. Existing research has employed various DL networks, such as

LSTM [27], GRU [28], [29], and GNN [30], to model packet length sequences. For instance, FS-Net [28] uses a bidirectional GRU to encode packet length sequences and introduces a reconstruction mechanism to ensure the effectiveness of the learned features. ERNN [27] is a model specifically designed for network-induced phenomena, utilizing LSTM networks to classify packet length sequences. Han et al. [30] constructed the graphs based on packet length sequences using GNN to enhance detection capabilities. Packet length sequences have been proven effective for encrypted traffic classification. However, length features are relatively coarse-grained, leading to the loss of detailed information. This makes it less suitable for fine-grained classification in some scenarios and challenging to distinguish between attacks with similar feature distributions.

Therefore, some works have utilized raw byte sequences for intrusion detection in encrypted traffic. For example, Wang et al. [33], Dong et al. [34], and Maonan et al. [35] transformed the hexadecimal byte sequences of encrypted flows into grayscale images and employed CNN models to classify them. Meng et al. [36] used a Transformer to encode the byte sequences of packets and then applied natural language processing techniques for classification. Gao et al. [37] represented the discrete hexadecimal bytes as words and incorporated an attention mechanism into CNN and BiLSTM architectures to improve intrusion detection accuracy by focusing on key information and capturing sequential patterns. Zhao et al. [38] introduced a traffic classification method based on a masked autoencoder (MAE), employing a multilevel flow representation and a traffic Transformer model to better extract features from raw byte data.

The aforementioned algorithms rely solely on single features, which may lead to several limitations. First, they are unable to comprehensively characterize the complex behavioral patterns of network traffic, potentially omitting multidimensional information. Second, these methods are vulnerable to evasion, as adversaries can manipulate single features (e.g., altering packet lengths or byte distributions) to circumvent detection. Finally, due to the limited information captured, the performance of the models often reaches a bottleneck, posing significant challenges for further enhancement.

C. Multimodal Models

Multimodal learning can enhance performance by capturing patterns from multiple perspectives and addressing the limitations of single-feature representations. Kong et al. [39] employed a sliding sampling window to conduct real-time sampling of network conversations and extracted three sequential features—bytes, lengths, and time intervals—to capture the patterns of different types of traffic. He et al. [40] proposed a novel multiview solution to reduce feature complexity and explored the use of multimodal DL to construct effective feature fusion modules for learning the underlying structure of traffic data. Yu et al. [41] designed a DL method based on multiview learning, utilizing an ensemble classifier for multiclass classification of minority attacks. Yun et al. [42] extracted three attribute sequences from TLS flows and used DL models to capture long-term dependencies among elements in packet sequences. Tan et al. [43] combined packet length

sequence features with statistical features to construct interflow spatiotemporal correlation graphs, uncovering specific spatiotemporal correlations among network flows. Wang et al. [44] proposed a multimodal encrypted traffic classification framework, AppNet, which uses a 1D-CNN to extract features from the first 1014 B of the initial packet and an LSTM to learn temporal relationships in packet length sequences, ultimately concatenating features learned from both perspectives for classification.

D. Real-Time Models

IoT involves a vast number of devices, frequently handling critical data and sensitive information. IDSs must monitor network traffic and device behavior in real time to promptly identify and respond to security threats. Tong and Zhang [1] employed an improved residual temporal convolutional network to achieve efficient spatiotemporal feature extraction and incorporated a channel attention mechanism to optimize computational efficiency. With a training time of 52 s, their model is claimed to support real-time detection. Alrefaei and Ilyas [45] reduced the prediction time to 0.0311 s by leveraging feature selection and an RF classifier, significantly enhancing detection efficiency while maintaining accuracy. Wang et al. [46] proposed a HyperDetect, which utilizes brain-inspired hyperdimensional computing (HDC) to achieve a $31.83\times$ improvement in inference speed, offering a feasible real-time intrusion detection solution for resource-constrained IoT devices. Yan et al. [47] introduced a node selection strategy based on clustering, which reduced the average training time of the model to 81.76 s, significantly accelerating training while also improving overall accuracy. While these methods focus on improving training and testing efficiency through model optimization and can meet low-latency online detection needs, they lack real-time processing mechanisms (e.g., time windows) for handling continuous and dynamic online traffic, making them less practical for real-world applications.

III. PRELIMINARIES

A. Granularity of Network Traffic

Network traffic is composed of a set of consecutive and ordered packets. To analyze it, a discrete representation of the traffic is required, which refers to the granularity of network traffic identification. Common granularities of network traffic mainly include packets, flows, and channels, which are defined as follows.

1) *Packet*: A packet is a fundamental unit of network traffic, composed of a series of hexadecimal bytes, including the header and payload. In encrypted traffic, the payload of the packet is encapsulated by encryption protocols, making it invisible externally, thereby ensuring the security and privacy of user information. A packet p_i is defined as follows, where a_i represents the five-tuple information (source IP, source port, destination IP, destination port, and protocol), t_i is the timestamp, i.e., the capture time, and c_i is the payload

$$p_i = (a_i, t_i, c_i). \quad (1)$$

2) *Flow*: A flow f_i refers to an ordered set of all packets that share the same or bidirectional five-tuple information. It is defined as follows, where P_f^i comprises multiple packets

arranged in temporal order, i.e., $t_1 < \dots < t_m$

$$f_i = (a_i, P_f^i) \\ P_f^i = \{p_1, \dots, p_m\} = \{(a_1, t_1, c_1), \dots, (a_m, t_m, c_m)\}. \quad (2)$$

3) *Channel*: A channel cn_i represents the traffic between a pair of IP addresses, i.e., an ordered set of packets exchanged between two hosts. It includes multiple network flows with the same or reversed IP pairs. It is defined as follows, where e_i denotes the IP pair information (source IP and destination IP), and $t_1 < \dots < t_n$

$$cn_i = (e_i, P_{cn}^i) \\ P_{cn}^i = \{p_1, \dots, p_n\} = \{(a_1, t_1, c_1), \dots, (a_n, t_n, c_n)\}. \quad (3)$$

Existing IDSs typically employ a single flow as the granularity for detection. In contrast, this article conducts intrusion detection at the channel granularity, which allows for a more comprehensive learning of rich behavioral features. For instance, in a port scanning attack, the attacker uses automated tools to perform large-scale port scans on the target host, aiming to identify open ports and exploit vulnerabilities to launch attacks. During this process, each packet sequence between the attacker and each requested port on the target host constitutes a flow. At this point, it is challenging to determine whether a single flow represents malicious or benign traffic solely based on individual flows. In comparison, observing at the channel granularity—that is, without distinguishing between ports and considering all packets sent by the attacker to the target host over a period of time—allows for a more holistic analysis and facilitates easier identification of such attacks.

B. Transformer

This article employs the Transformer encoder architecture, which primarily includes the multihead self-attention mechanism, position embedding, add and norm layers, and feedforward networks.

1) *Multihead Self-Attention Mechanism*: The essence of the attention function can be described as a mapping from a query (Q) to a set of key-value (K - V) pairs. In self-attention, the Q , K , and V are derived from the same input vector x through three distinct linear transformation matrices W^Q , W^K , and W^V . The multihead self-attention mechanism projects the input into different representation subspaces using multiple sets of weight matrices (Transformer uses eight attention heads). The inference process is as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (4)$$

$$\text{head}_i = \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right) \quad (5)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O. \quad (6)$$

Here, d_k is the dimension of K , and h is the head number of self-attention. Different heads can be understood as multiple independent and parallel self-attention mechanisms, each focusing on different semantics within the sequence. For the byte sequences in this article, they can focus on different parts

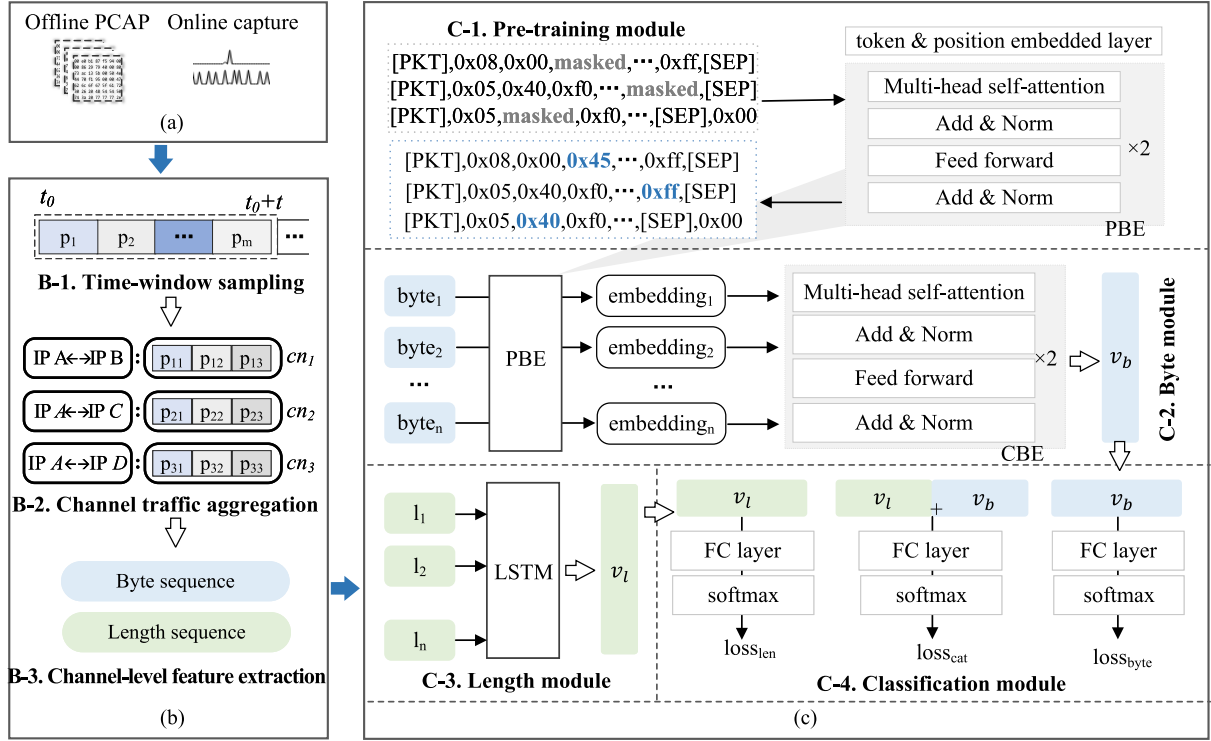


Fig. 2. System framework of the proposed method, consisting of three components. (a) Data collection for collecting raw network traffic, (b) data preprocessing that generates both channel-level byte sequence and packet length sequence features, and (c) multimodal learning where a pretraining module performs unsupervised learning on byte sequences, followed by byte and length modules processing respective features, with a classification module concatenating both features for final prediction.

within a packet and the relationships between different packets in a channel traffic.

2) *Position Embedding:* The self-attention mechanism is inherently permutation-invariant, meaning that shuffling the order of the input sequence does not change the output attention weights. Therefore, the Transformer superimposes input data with position information through position embedding, which enables the model to distinguish words at different positions. In network packets, the fixed positions of bytes correspond to specific fields that carry key information, making position embedding essential.

3) *Add and Norm:* The add and norm layer consists of two components: add and norm, representing residual connection and layer normalization operations, respectively. It is applied to both the multihead self-attention layer and the feedforward network, as expressed in the following formula:

$$\begin{aligned} & \text{LayerNorm}(X_{\text{embed}} + \text{MultiHeadAttention}(X_{\text{embed}})) \\ & \text{LayerNorm}(X_{\text{hidden}} + \text{FeedForward}(X_{\text{hidden}})) \end{aligned} \quad (7)$$

where X_{embed} and X_{hidden} denote the inputs to the MultiHeadAttention and FeedForward, respectively.

4) *Feedforward Network:* The feedforward network is a two-layer neural network that maps the vectors obtained from multihead attention to a higher dimensional space. It then applies the ReLU nonlinear activation function for feature selection, before projecting the results back to the original dimension. The formula is as follows:

$$\text{FeedForward}(X_{\text{hidden}}) = \text{ReLU}(w_1 X_{\text{hidden}} + b_1) w_2 + b_2 \quad (8)$$

where w_1 , b_1 , w_2 , and b_2 represent the parameters of the two linear transformations.

IV. METHODOLOGY

This article proposes a real-time channel-level IDS based on multimodal learning, which consists of three stages: data collection, data preprocessing, and multimodal learning. Fig. 2 shows the system framework of the proposed method. In implementation, the method first performs time-window-based sampling and preprocessing on the input traffic, extracting channel-level byte sequences and packet length sequences as two modal features. These features are then processed by their respective modules, the byte and length modules, to learn embedded representations. The resulting representations are concatenated and fed into the classification layer, which produces the final decision. Fig. 3 further details the specific workflow of the above process.

A. Data Collection

This solution supports intrusion detection in both online and offline scenarios. In the online detection mode, packets arriving at the network interface are captured in real time. In the offline detection mode, packets are read from PCAP files for subsequent analysis.

B. Data Preprocessing

This article introduces a novel preprocessing method based on time-window segmentation, which includes three

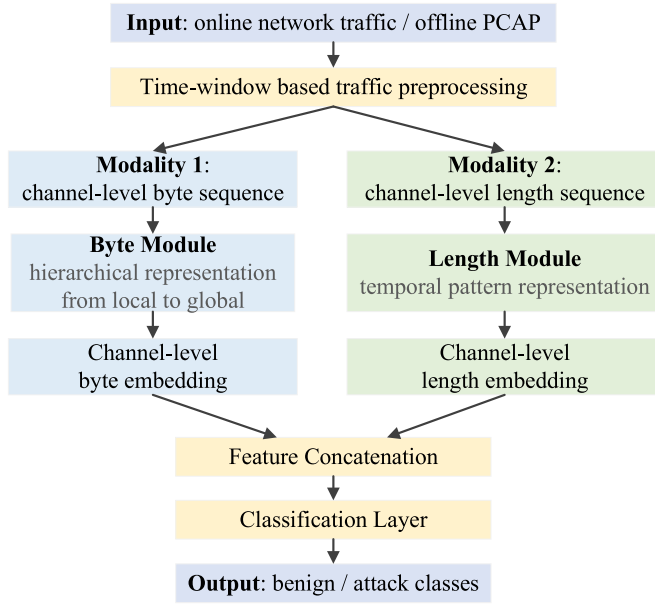


Fig. 3. Workflow of the proposed method.

components: time-window sampling, channel traffic aggregation, and channel-level feature extraction.

Algorithm 1 Online Network Traffic Preprocessing Algorithm

Input: Network traffic trace (NTT), time-window (t), packets/sample (n), bytes/packet (p)

Output: List of samples (X)

```

1: procedure PREPROCESS( $NTT, t, n, p$ )
2:    $t_0 \leftarrow -1$   $\triangleright$ Initialize the time window start-time
3:   for  $pkt$  in  $NTT$  do  $\triangleright$ Loop over the packets
4:      $id \leftarrow pkt.ip\_pairs$   $\triangleright$ Extract the IP pairs as id
5:     if  $t_0 == -1$  or  $pkt.time > t_0 + t$  then
6:        $t_0 \leftarrow pkt.time$   $\triangleright$ Start the new time window
7:     end if
8:     if  $len(X[t, id]) < n$  then  $\triangleright$ Max  $n$  packets
9:        $pkt.features \leftarrow (pkt\_len, pkt\_byte)$ 
10:       $X[t, id].append(pkt.features)$ 
11:    end if
12:  end for
13:   $X \leftarrow padding(X, n, p)$   $\triangleright$ Unify the feature length
14:  return  $X$ 
15: end procedure
    
```

1) *Time-Window Sampling*: The proposed preprocessing mechanism applies to both online real-time captured traffic data and offline stored PCAP files, collectively referred to as network traffic trace (NTT). For packets arriving in chronological order, we use a time window t to capture all packets falling within the interval $[t_0, t_0 + t]$, where t_0 represents the timestamp of the first packet captured within that time window. This process is described in lines 5 and 6 of Algorithm 1. This sampling method generates observations consistent with the traffic collected by existing online systems, providing reliable data for simulating real-time attack detection in offline scenarios, which can be used for subsequent model training. Most importantly, this sampling approach is key to enabling

real-time detection in online scenarios. Specifically, we capture packets online in units of time windows. Once a time window expires, all packets within that window are immediately sent for analysis without waiting for the complete network flow.

2) *Channel Traffic Aggregation*: For the packets within a time window, we group them at the channel granularity. The source IP and destination IP of the packets are extracted as channel identifiers (see line 4 of Algorithm 1). It is important to note that channel identifiers are bidirectional, meaning that packets within each channel share the same or reversed IP pairs.

3) *Channel-Level Feature Extraction*: For each packet, its payload length and raw bytes are extracted as features. The features of multiple packets within a channel form length and byte sequences. For encrypted traffic, the intuition behind using length sequences to represent packets lies in the fact that different types of network traffic exhibit distinct length feature sequences over time due to their inherent behavioral and pattern differences. For example, DDoS attack traffic usually surges sharply in a short period, resulting in a long sequence with significant spikes and fluctuation patterns. In contrast, normal traffic tends to have a more stable and uniform length sequence. Although length sequences have been proven effective for encrypted traffic classification, relying solely on packet lengths can lead to the loss of many details. The discriminative power of packet length sequences is inherently limited when they exhibit high similarity. As confirmed in Section V-A4, different types of network flows can share identical packet length sequences, and this feature overlap creates significant ambiguity for the model.

Therefore, this article also adopts more fine-grained byte sequence features, leveraging multimodal DL methods to classify encrypted traffic. Multimodal learning can enhance performance by capturing patterns from multiple perspectives and mitigating the limitations of single-feature representations. Although the content of encrypted flows cannot be directly interpreted, the distribution of bytes in packets varies across different network behaviors, and certain key information in the packet headers can provide discriminative power for detection tasks. By combining packet lengths and raw byte data, this method can better integrate channel traffic features, effectively learning sequential relationships from byte-to-byte and packet-to-packet.

Assume a channel sample is denoted as $cn_i = \{p_1, p_2, \dots, p_m\}$, where p_j is the j th packet in this sample. For p_j , the byte sequence $byte_j = [b_1, b_2, \dots, b_l]$ is extracted, where b_l is the last byte and $l = len(p_j)$ is the length of packet p_j . After extracting these two features for all packets in cn_i , they are concatenated to obtain channel-level byte sequence features and length sequence features. To standardize the length, we select the first n packets of each channel and the first p bytes of each packet, padding with zeros if necessary. Therefore, the channel sample cn_i yields the byte sequence feature

$$\begin{aligned}
 B_i &= [byte_1, byte_2, \dots, byte_n] \\
 &= [[b_{11}, b_{12}, \dots, b_{1p}], [b_{21}, b_{22}, \dots, b_{2p}], \\
 &\quad \dots, [b_{n1}, b_{n2}, \dots, b_{np}]]
 \end{aligned} \tag{9}$$

and the length sequence feature

$$L_i = [l_1, l_2, \dots, l_n]. \quad (10)$$

To prevent IP and MAC addresses from influencing model training and detection results, they are randomly masked, i.e., set to zero. The process of channel-level feature extraction and padding corresponds to lines 8–13 of Algorithm 1.

C. Multimodal Learning

We design an end-to-end multimodal DL network, consisting of four modules: pretraining, byte, packet length, and classification. First, the pretraining module learns a byte embedding model, with the Transformer serving as the core component. Its multihead self-attention mechanism enables the learning of relationships among bytes. Second, the byte module uses the pretrained model during training to obtain packet-level embedding representations. Then, another Transformer is employed to learn the relationships among different packet embeddings, thereby obtaining channel-level embedding representations. Third, the length module models the packet length sequence using LSTM to learn complementary features from the second modality. Finally, the classification module concatenates the hidden features output by the byte and length modules to perform fusion and classification.

1) *Pretraining Module*: The objective of the pretraining module is to learn the relationships among bytes within a packet through unsupervised pretraining, thereby acquiring the ability to encode and represent packet bytes. The pretraining data consists of raw byte sequences at the packet level. Specifically, we use Scapy to parse PCAP files, extract the payload of each packet, and truncate it to the first p bytes. Payloads shorter than p bytes are zero-padded to this fixed length. Next, special tokens are added to each processed byte sequence to facilitate model learning. A [PKT] token is placed at the beginning, and a [SEP] token is added at the end, resulting in the final input sample $X_{\text{input}} = [\text{PKT}, b_1, b_2, \dots, b_p, \text{SEP}]$. We choose the Transformer as the pretraining model because its self-attention mechanism allows each byte to incorporate information from other bytes. Inspired by the BERT [48] model, during pretraining, we randomly mask 15% of the packet bytes and continuously learn from adjacent bytes, with the goal of enabling the Transformer network to recover the masked bytes. We refer to this Transformer as the packet-level byte encoder (PBE), which produces a comprehensive byte embedding for the entire packet by integrating contextual information from all bytes. The overall pretraining procedure is outlined in Algorithm 2.

2) *Byte Module*: The pretrained PBE is leveraged to obtain a byte embedding for each packet. Given the inherent correlations between adjacent packets in a traffic flow, we design the byte module to model the interpacket relationships. The task of the byte module is to learn and represent the 2-D byte sequence features, generating a 1-D channel-level byte embedding representation. We employ another Transformer network to process the byte embeddings of all packets (from PBE) within the channel sample cn_i , resulting in a channel-level byte embedding representation v_b^i . We refer to this Transformer model as

Algorithm 2 Pretraining Algorithm

Input: Packet Capture Data File (PCAP), sequence length p , mask ratio m

Output: Trained PBE

```

1: procedure PREPROCESS(PCAP,  $p, m$ )
2:   for each packet in PCAP do
3:     Extract payload
4:     Truncate/zero-pad payload to fixed length  $p$ 
5:     Add [PKT] and [SEP] tokens  $\rightarrow X_{\text{input}}$ 
6:     Randomly mask  $m\% \rightarrow X_{\text{masked}}$  and  $Y_{\text{target}}$ 
7:   end for
8:   Feed  $X_{\text{masked}}$  to PBE  $\rightarrow Y_{\text{pred}}$ 
9:   Compute cross-entropy loss between  $Y_{\text{pred}}$  and  $Y_{\text{target}}$ 
10:  Update PBE via backpropagation
11:  return Trained PBE
12: end procedure

```

the channel-level byte encoder (CBE). Specifically, for the byte sequence feature $B_i = [\text{byte}_1, \text{byte}_2, \dots, \text{byte}_n]$ corresponding to the channel sample cn_i , where byte_j represents the byte sequence of the j th packet, the channel-level byte embedding obtained after encoding by the CBE can be expressed as

$$v_b^i = \text{CBE}(\text{PBE}(\text{byte}_1), \text{PBE}(\text{byte}_2), \dots, \text{PBE}(\text{byte}_n)). \quad (11)$$

3) *Length Module*: For the length sequence feature $L_i = [l_1, l_2, \dots, l_n]$ corresponding to the channel sample cn_i , we employ a Bi-LSTM network to learn the underlying relationships among packets. The hidden state of the last time step is used as the representation result v_l^i , which encapsulates summary information of the entire input sequence

$$v_l^i = \text{LSTM}(L_i). \quad (12)$$

4) *Classification Module*: First, we concatenate the representation results from the byte module and the length module, and then pass them through a fully connected layer for classification. These two features are complementary. On the one hand, the fine-grained byte information supplements the coarse-grained length information. On the other hand, since the first p bytes are truncated to standardize the length, the byte information cannot reflect the complete length information of the packets. Therefore, the packet length information also serves as a supplement to the byte information. This concatenation-and-classification approach allows the model to better integrate these two features and uncover their underlying relationships. Subsequently, to further optimize the representation capabilities of the two modules, we classify the representation results from each module separately to assist in the final decision-making.

Thus, the overall network's loss function consists of three parts: the classification loss of the byte module $\text{loss}_{\text{byte}}$, the classification loss of the length module loss_{len} , and the final classification loss loss_{cat}

$$\text{loss}_{\text{byte}} = \frac{1}{N} \sum_{i=1}^N H(y_i, \text{PFC}_{\text{byte}}(v_b^i)) \quad (13)$$

$$\text{loss}_{\text{len}} = \frac{1}{N} \sum_{i=1}^N H(y_i, p_{\text{FC_len}}(v_l^i)) \quad (14)$$

$$\text{loss}_{\text{cat}} = \frac{1}{N} \sum_{i=1}^N H(y_i, p_{\text{FC_cat}}(\text{Concat}(v_b^i + v_l^i))). \quad (15)$$

Here, N is the number of training samples, $H(\cdot)$ is the cross-entropy loss function, and y_i is the true label of the i th sample. $\text{Concat}(\cdot)$ represents the concatenation operation of two vectors. $p_{\text{FC_byte}}$, $p_{\text{FC_len}}$, and $p_{\text{FC_cat}}$ denote the predicted probabilities of the byte features, length features, and combined features, respectively, output by the three fully connected layers.

The independent losses learned by the two modules ensure that the features of each modality are completely learned, while the joint loss captures the associations between two modalities. The combination of independent losses and joint losses not only preserves the independence of the features but also promotes their complementarity. This multiloss joint training approach guides the model to converge faster and learn the behavioral patterns of channel traffic more comprehensively, thereby enhancing overall performance.

V. EVALUATION

A. Experimental Preparation

1) *Datasets*: To validate the adaptability of our method to different IoT scenarios and its consistency in both simulated and real-world environments, we conduct evaluations on three publicly available datasets and one self-collected dataset.

- 1) *MQTT-IoT-IDS2020* [49]: This dataset focuses on the MQTT protocol, one of the core communication protocols among IoT devices. By simulating real-world network environments, it covers normal operations and four types of attack scenarios, thereby facilitating the assessment of our method's adaptability and detection capabilities under simulated environments.
- 2) *Ton-IoT* [50]: This dataset is based on real-world IoT device network traffic and incorporates nine attack patterns alongside normal behavior patterns. Its diversity allows for the validation of our method's generalizability across different IoT devices and network environments.
- 3) *CICIoMT2024* [51]: This dataset focuses on the Internet of Medical Things (IoMT) scenarios, containing five types of network attacks targeting 40 IoMT devices and covering multiple protocols, such as Wi-Fi, MQTT, and Bluetooth. By combining real medical devices and simulated attacks, this dataset can verify our method's applicability in complex scenarios within specific industries.
- 4) *Self-Collected Dataset*: The self-collected dataset is based on a small-scale IoT network comprising devices such as Raspberry Pi, smart bulbs, temperature and humidity sensors, and cameras. Specifically, the network topology is illustrated in Fig. 4. The network employs hybrid Wi-Fi and Ethernet connectivity, with the Raspberry Pi serving as the attack initiator to capture traffic data using tcpdump. The dataset comprises normal traffic along with four attack types: ACK flooding,

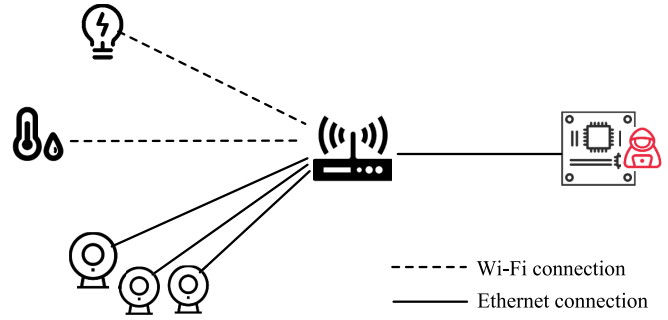


Fig. 4. Network topology of the self-built small-scale IoT testbed, containing Raspberry Pi (attack initiator), smart bulbs, temperature/humidity sensors, and cameras, with hybrid Wi-Fi and Ethernet connections.

TABLE II
COMPOSITION AND SPLITTING OF ALL DATASETS

Dataset	Category	Sample Count		
		Training	Validation	Test
MQTT-IoT IDS2020	benign	10245	2467	3078
	scan_A	6449	1613	2068
	scan_sU	6777	1687	2115
	mqtt_bruteforce	4893	1208	1601
	sparta	7693	1860	2406
Ton-IoT	benign	8803	2176	2797
	injection	6460	1605	2081
	MITM	6650	1680	2109
	backdoor	24896	6238	7726
	DDoS	6415	1599	1992
	DoS	6398	1629	1986
	ransomware	6442	1620	1967
	scanning	6432	1548	2042
	XSS	7754	1992	2409
CICIoMT2024	password	7353	1814	2267
	benign	6406	1570	2024
	DDoS	5577	556	661
	DoS	6345	1627	2028
	Recon	4163	1054	1302
	MQTT	2074	527	579
Self-collected dataset	Spoofing	6397	1569	2034
	benign	2405	284	311
	ACK flooding	2490	274	289
	SYN flooding	2454	255	291
	brute force	815	107	119
	port scanning	1028	113	104

SYN flooding, brute-force, and port scanning, containing 1 37 396, 1 41 709, 3 13 462, 3 18 209, and 99 402 packets, respectively. The complete dataset will be made publicly available along with the code of this article to facilitate future verification and research.

All datasets are partitioned into a training set and a test set at a 9:1 ratio. A validation set is then subsampled from the training set using the same ratio for model training. Detailed sample distributions are shown in Table II. The self-collected dataset, although smaller in scale, serves to evaluate the model's adaptability in real-world few-sample scenarios.

2) *Experimental Setup*: In the offline experiments, we train and evaluate the model using PyTorch on a server equipped with an NVIDIA RTX 3090 GPU. The Transformer models used in the pretraining module and the byte module both have eight heads and two layers, with a byte embedding dimension of 128 in the pretraining module. The length module employs

TABLE III
EFFECT OF PACKET LENGTH SEQUENCE OVERLAP

Dataset	Overlap Rate	Before Removal		After Removal		ACC ↑	F1 ↑
		ACC	F1	ACC	F1		
MQTT-IoT	25%	62.79	62.99	78.67	78.41	25.29%	24.48%
Ton-IoT	11%	52.34	45.31	57.30	52.12	9.47%	15.02%
CICIoMT2024	39%	41.45	33.06	62.80	58.18	51.50%	75.98%

a bidirectional LSTM with a length embedding size of 32 and a hidden state size of 128. In addition, unless otherwise specified, the following experiments adopt a default time window of $t = 1$ s, a maximum packet count of $n = 30$ per channel sample, and a byte truncation length of $p = 100$ per packet.

3) *Evaluation Metrics*: To assess the classification performance of the model, the following five well-known metrics are adopted: accuracy (ACC), precision [positive predictive value (PPV)], recall [true positive rate (TPR)], and $F1$ score ($F1$). The $F1$ score provides a comprehensive measure of model performance, representing the harmonic mean of PPV and TPR. The formal definitions of these metrics are as follows:

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad (16)$$

$$PPV = \frac{TP}{TP + FP} \quad (17)$$

$$TPR = \frac{TP}{TP + FN} \quad (18)$$

$$F1 = 2 \times \frac{PPV \times TPR}{PPV + TPR} \quad (19)$$

where TP = true positives, TN = true negatives, FP = false positives, and FN = false negatives. In addition, we also evaluate the detection efficiency and time consumption of the models by considering the number of model parameters, training and testing times, as well as the detection latency.

4) *Analysis of Packet Length Sequence Similarity*: As noted in Section IV-B3, “the discriminative power of packet length sequences is inherently limited when they exhibit high similarity.” In this section, we examine whether different classes of network flows share identical packet length sequences across three public datasets, and quantify the impact on classification performance. Table III reports the proportion of samples that have the same packet length sequence yet belong to different classes, and compares the classification performance of an LSTM model before and after removing such overlapping samples.

Results show that all three datasets exhibit sequence overlap to varying degrees, with the CICIoT2024 dataset showing a particularly high proportion of such samples at 39%. More importantly, the presence of this phenomenon markedly degrades classification performance because these ambiguous samples blur the decision boundary during training. After their removal, both accuracy and $F1$ score improve significantly. These findings demonstrate that relying solely on packet length sequences as a single modality is insufficient, which motivates our additional use of byte sequences.

B. Comparison With State-of-the-Art Methods

To evaluate the performance of our proposed method, we conduct comparative experiments using several state-of-the-art approaches. From the perspective of the features used, the following comparative experiments are selected: ERNN and Fs-Net utilize length sequence features, YaTC and 1D-CNN use byte sequence features, while AppNet and iDetector employ both length and byte features for multimodal learning. We reproduce these methods, with AppNet and iDetector being implemented based on the detailed descriptions in their respective papers, since their code was not publicly available. The implementation codes for all comparative experiments will also be made public in the future.

- 1) ERNN [27] uses LSTM to classify encrypted traffic based on packet length sequences. In our implementation, we do not consider the network-induced phenomenon mentioned in this article, setting $x = 1$ in the transition matrix.
- 2) Fs-Net [28] employs a bidirectional GRU to encode features from packet length sequences and introduces a reconstruction mechanism in the autoencoder to ensure the effectiveness of the learned features.
- 3) YaTC [38] uses a Transformer to pretrain on randomly masked network flow byte matrices, learning multilevel features at both packet and flow levels, followed by fine-tuning with labeled data.
- 4) 1D-CNN [33] converts sequences of hexadecimal bytes from encrypted network flows into grayscale images and utilizes a 1D-CNN model to classify these images.
- 5) AppNet [44] extracts features from byte sequences using a 1D-CNN and learns temporal relationships in packet length sequences using LSTM. Finally, the features learned from both views are concatenated for classification.
- 6) iDetector [39] extracts three sequence features of byte, length, and time interval from channel traffic, and classifies them using a lightweight deep neural network model called EdgeNet.

Table IV presents the classification results and model parameter counts of all methods on the four datasets. As shown in the results, our proposed model achieves the best classification performance across all datasets, demonstrating its robust generalization capability and adaptability to diverse IoT scenarios. Particularly on the three public datasets, the accuracy of RCML-IDS reaches around 98%. On the dataset collected from the real-world environment, the performance of additional models dropped significantly, with the accuracy of YaTC and iDetector decreasing by more than 10%. In contrast, our method still maintains a relatively high accuracy of 93%, indicating that the proposed approach exhibits strong robustness when facing real-world IoT environments. The slightly lower performance on the self-collected dataset compared to the public datasets may be primarily due to the limited number of samples for certain attack categories in the self-collected dataset, which affects the model’s performance.

Among the comparison methods, ERNN and Fs-Net, which only use length sequence features, perform the worst.

TABLE IV
CLASSIFICATION RESULTS OF ALL METHODS ON THE FOUR DATASETS

Model	Params	MQTT-IoT-IDS2020				Ton-IoT				CICIoMT2024				Self-collected dataset			
		ACC	PPV	TPR	F1	ACC	PPV	TPR	F1	ACC	PPV	TPR	F1	ACC	PPV	TPR	F1
ERNN	0.06M	67.10	73.66	67.10	66.92	71.32	71.88	71.32	70.73	53.07	52.74	53.07	48.54	65.89	67.32	65.89	63.30
Fs-Net	3.35M	73.79	77.18	73.79	72.70	78.26	78.43	78.26	78.16	54.94	58.64	54.94	52.03	69.63	69.40	69.63	68.43
1D-CNN	5.82M	83.74	83.60	83.74	83.52	84.79	84.92	84.79	84.69	84.42	85.76	84.42	84.76	77.02	77.34	77.02	77.05
YaTC	1.86M	94.50	95.29	94.50	94.30	93.07	93.11	93.07	93.06	95.42	95.50	95.42	95.44	84.51	85.49	84.51	84.27
AppNet	2.30M	93.92	94.00	93.92	93.78	94.60	94.73	94.60	94.62	94.66	94.94	94.66	94.74	89.05	89.44	89.05	88.93
iDetector	1.23M	92.85	92.76	92.85	92.60	93.30	93.41	93.30	93.26	93.82	93.93	93.82	93.82	81.60	81.67	81.60	81.56
RCML-IDS	2.15M	98.76	98.77	98.76	98.76	97.30	97.30	97.30	97.29	98.33	98.34	98.33	98.33	93.36	93.39	93.36	93.37

Particularly on the CICIoMT2024 dataset, their classification accuracy is inferior to 55%. This indicates that using only coarse-grained length sequence features for encrypted traffic classification is insufficient. The limited information contained in these features makes it difficult to accurately distinguish malicious network behaviors.

In contrast, byte sequences help models learn more comprehensive and detailed features. Notably, the YaTC method even outperforms the two multimodal DL models, AppNet and iDetector. This is attributed to its use of a Transformer to capture feature information from raw bytes at both the packet and flow levels, allowing for a more comprehensive consideration of traffic hierarchical relationships. This aligns with our design philosophy. However, YaTC directly stacks the bytes of multiple adjacent packets into a matrix and learns them as a whole. Our approach first learns at the packet level and then integrates packet-level embeddings to obtain a holistic representation of the entire channel traffic. Compared to YaTC, our hierarchical design is better able to capture both the internal features of packets and the relationships between packets. However, the 1D-CNN method, owing to its simpler byte processing approach and single model structure, attains a classification accuracy of around 84% on public datasets.

Among the two multimodal learning models, AppNet also utilizes byte sequences and packet length sequences as features. However, for the byte sequences, it only employs a 1D-CNN, which has weaker capabilities in modeling long-range dependencies and global sequential relationships. The iDetector uses three modal features: byte, packet length, and time intervals, and applies a nonlinear transformation method to convert them into fixed-size three-channel features. Specifically, their proposed nonlinear transformation maps the values of packet length and time intervals to the range of 0–255 and expands the packet length and time-interval sequences into 2-D vectors. We argue that this discretization approach loses details of the specific values, especially for continuous features, like packet length and time intervals, thereby leading to inferior detection performance compared to AppNet and our method.

In addition, Fig. 5 illustrates the comparison of computational efficiency across all evaluated methods. Notably, our method exhibits the longest training time, which is attributed to the computational complexity associated with learning byte-level features. Nevertheless, leveraging the parallel computing advantages inherent in the multihead self-attention mechanism

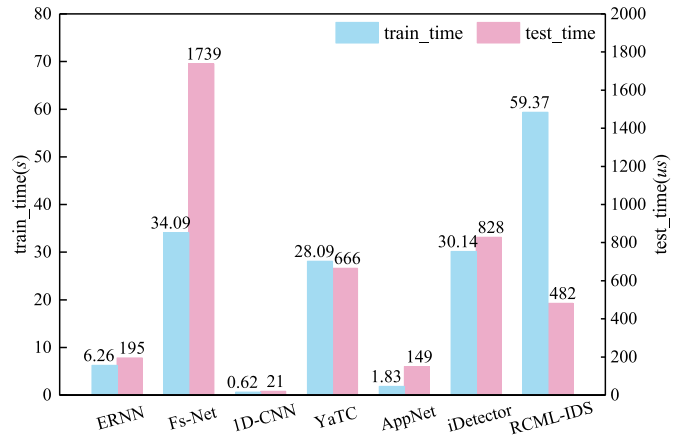


Fig. 5. Comparison of computational efficiency among all methods on the MQTT-IoT-IDS2020 dataset (training time per 100 batches versus test time per sample). The training time of YaTC and RCML-IDS represents the sum of the pretraining time and fine-tuning/formal training.

of the Transformer architecture, our approach achieves leading accuracy while maintaining moderate inference speeds compared to other methods.

C. Ablation Experiment

To investigate the impact of each module in our model, we conduct ablation experiments on the Ton-IoT dataset from the following aspects.

- 1) *Pretraining Module*: We remove the pretraining process and initialize the parameters of the PBE in the byte module randomly. This variant is denoted as RCML-nopre.
- 2) *Byte Module*: We remove the byte module and use only the packet length feature without byte sequence features. This variant is denoted as RCML-len.
- 3) *Length Module*: We remove the length module and use only the byte sequence feature without packet length features. This variant is denoted as RCML-bytes.
- 4) *Classification Module*: In the original design, the classification module combines the losses from the byte feature classification, length feature classification, and the concatenated classification. To validate its effectiveness, we conduct an ablation experiment using only

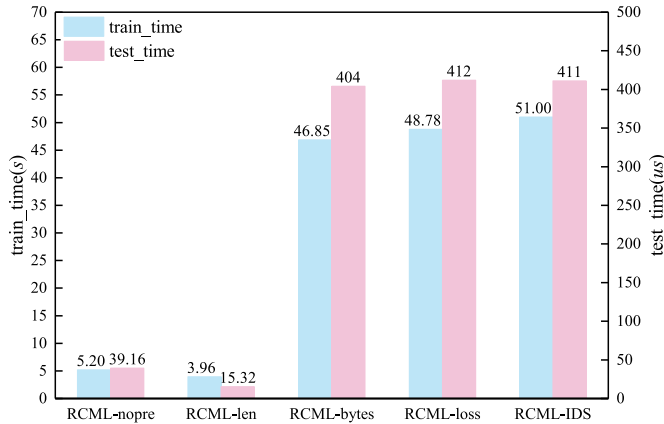


Fig. 6. Effect of each module on computational efficiency. The training time of RCML-bytes, RCML-loss, and RCML-IDS represents the sum of the pretraining time and formal training.

the final concatenated classification loss. This variant is denoted as RCML-loss.

The analysis of the Table V and Fig. 6 leads to the following conclusions.

- 1) *Impact of Pretraining*: Omitting the pretraining process significantly increases training and testing speeds but results in a 4% drop in accuracy. This indicates that pretraining enables the model to more effectively learn the relationships between bytes within packets, thereby enhancing overall performance.
- 2) *Feature Module Comparison*: Compared with using the byte module alone, relying solely on length features yields considerably poorer results (with an accuracy gap of over 10%), further validating the necessity of employing fine-grained byte features. However, combining both features through multimodal learning effectively leverages their complementary nature, improving classification accuracy by an additional 2%. However, the introduction of byte feature learning also brings significant computational overhead, leading to substantial increases in training and inference times.
- 3) *Effectiveness of Multiloss Training*: The multiloss joint training approach achieves a 1% higher classification accuracy compared to using only the single concatenated classification loss. This suggests that the multiloss training strategy helps the model better balance the relationship between byte and length features, thus improving detection effectiveness.

D. Sensitivity Analysis

This section analyzes the impact of two key parameters, n and p , on the MQTT-IoT-IDS2020 dataset.

1) *Maximum Number of Packets n Per Channel*: As illustrated in Fig. 7, the classification accuracy initially increases and then decreases as the number of packets ranges from 5 to 50, with fluctuations not exceeding 1%. The highest accuracy of 98.76% is achieved when $n = 30$. When n exceeds 30, the classification performance of the model declines. In addition, an increase in the number of packets leads to higher model

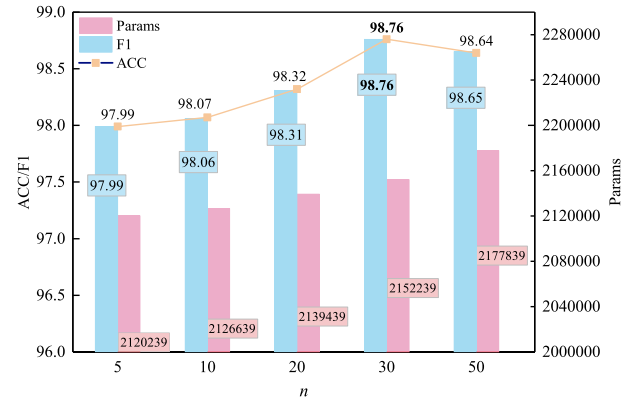


Fig. 7. Trends of ACC, F1, and the number of model's parameters with different n .

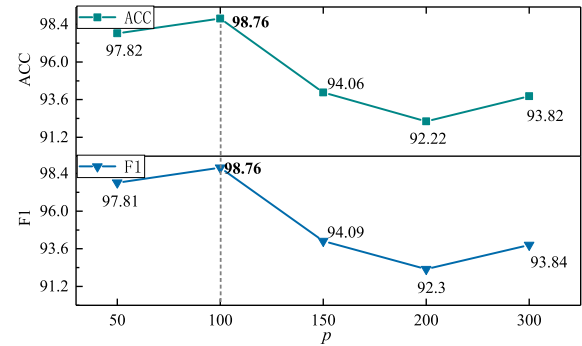


Fig. 8. Impact of byte size p on classification performance.

complexity, which in turn may result in longer training and testing times. Therefore, we set $n = 30$ as the optimal value.

2) *Number of Bytes p Per Packet*: We test the model with different packet truncation lengths, specifically $p = 50, 100, 150, 200$, and 300 B. In addition to classification performance metrics, we also record the training and testing times to comprehensively evaluate the model's performance and computational efficiency. The results in Fig. 8 show that truncating the first 100 B of each packet yields significantly better classification performance compared to other settings. As the number of truncated bytes increases, the classification performance deteriorates, which may be attributed to the fact that the latter bytes of the packet contain encrypted payloads (ciphertext). Since ciphertext typically appears as random or meaningless data, such information may introduce noise and interfere with the model's judgment. Furthermore, Fig. 9 demonstrates that both training and inference times scale linearly with byte count. Therefore, considering both classification performance and computational efficiency, $p = 100$ is the optimal choice, as it ensures high classification accuracy while maximizing computational efficiency.

E. Real-Time Detection Analysis

To verify that our proposed preprocessing method can monitor network interfaces in real-time and promptly output detection results, we deploy the model on a server equipped with an Intel² Xeon2 Silver 4214 CPU @ 2.20 GHz (without

²Registered trademark.

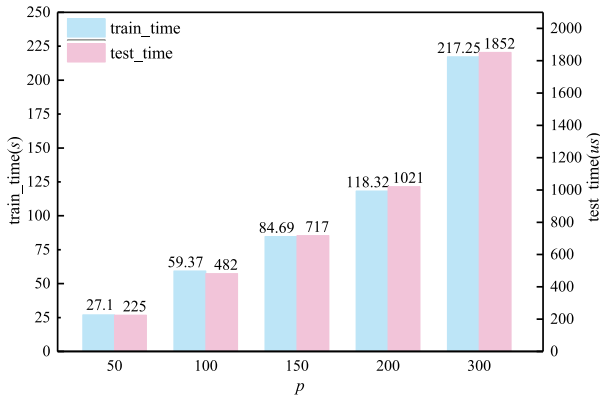
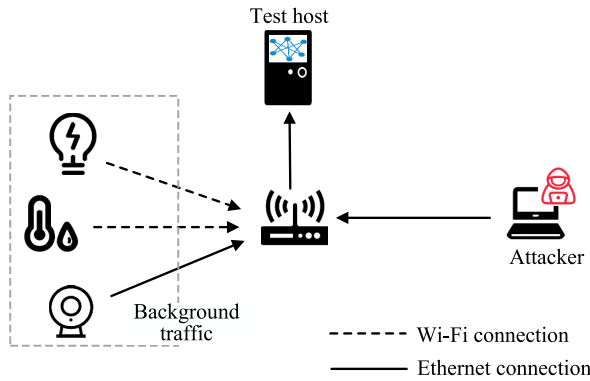
Fig. 9. Impact of byte size p on computational efficiency.

Fig. 10. Network topology for real-time testing, comprising smart devices generating background traffic and a PC replaying attack traces, with a test server capturing and detecting traffic.

GPU) and conduct real-time testing. The network topology of the test environment is illustrated in Fig. 10, where a small local area network is constructed, containing multiple smart devices to generate background traffic. In addition, a PC is used to replay attack traffic from the MQTT-IoT-IDS2020 dataset to simulate real-world network attack behaviors. The test server captured traffic in real-time using the tcpdump tool. Packets captured within a 1-s time window are preprocessed and immediately fed into the model for detection, with results outputted accordingly.

It should be noted that the core of traffic replay technology lies in simulating and reproducing existing traffic from PCAP files, which is inherently unidirectional. In the MQTT-IoT-IDS2020 dataset, the scan_A (active scanning attack) and scan_sU (UDP scanning attack) both involve the attacker unilaterally sending a large number of probe packets to the target device (UDP port), exhibiting strong unidirectional characteristics. In contrast, the Sparta SSH brute-force attack and MQTT brute-force attack require the target device to return response results, making their interaction process bidirectional and impossible to fully simulate using unidirectional traffic replay technology. Given this, the experiment adopts two types of scanning attacks for replay and real-time testing.

From the ablation experiments on each module in Section V-C, it is evident that omitting the pretraining module results in a decrease in the model's classification accuracy but

TABLE V
EFFECT OF EACH MODULE ON CLASSIFICATION PERFORMANCE

	ACC	PPV	TPR	F1	Params
RCML-nopre	92.98	93.12	92.98	92.97	1.71M
RCML-len	83.77	84.30	83.77	83.91	0.61M
RCML-bytes	95.05	95.20	95.05	95.07	1.54M
RCML-loss	96.28	96.34	96.28	96.25	2.19M
RCML-IDS	97.30	97.30	97.30	97.29	2.19M

TABLE VI
DETECTION PERFORMANCE AND COMPUTATIONAL COST FOR TWO ATTACK TYPES

Attack type	Scan_A		Scan_sU	
	RCML-light	RCML-IDS	RCML-light	RCML-IDS
ACC	97.87	98.80	96.94	98.94
TPR	97.87	98.80	94.06	98.94
F1	98.92	99.40	96.94	99.47
DPT	1.69	1.80	1.17	1.25
MIT	19.55	160.54	16.98	158.25

DPT: Data Processing Time ($ms/sample$);
MIT: Model Inference Time ($ms/sample$);

a significant improvement in testing speed. This configuration can be considered as our lightweight model (RCML-light). Consequently, we test their detection performance against the two attacks and record the key metrics in Table VI.

In terms of detection performance, RCML-IDS achieves approximately 99% accuracy and $F1$ score for real-time detection of both types of attacks. In contrast, RCML-light experiences a slight decline in detection accuracy. This comparison demonstrates the advantage of the preprocessing module, which can more precisely extract the relationships between bytes within packets. Regarding time consumption, since the data processing procedures of both models are identical, the time spent on data processing is similar. However, the lightweight model boasts a much faster inference efficiency, with an inference speed nearly eight times faster than the full model. The total processing time per sample, which includes both data processing and model inference time, is approximately 20 ms for the lightweight model and 160 ms for the full model. This trade-off between performance and efficiency indicates that RCML-light achieves a gain in real-time capability at the cost of acceptable performance degradation, making it suitable for the needs of most IoT applications, such as smart homes and environmental monitoring.

F. Comparison of Online Traffic Processing Methods

The iDetector method [39] also supports online real-time packet capture and detection. The key difference from our approach lies in its handling of incoming packets: iDetector does not employ time windows. Instead, it adds each arriving packet to the corresponding channel sample and waits until the sample contains ω packets before sending it for detection. However, this approach may result in longer waiting times, potentially leading to delayed analysis. Therefore, in this section, we compare the detection latency of our proposed

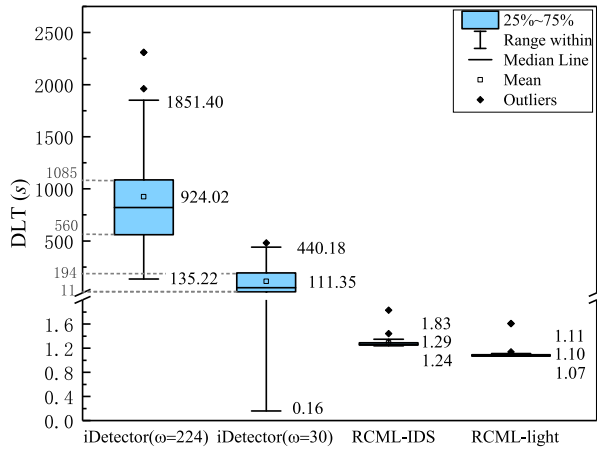


Fig. 11. Comparison of detection latency among online processing methods. Data markers represent (top to bottom): maximum, mean, and minimum values.

time-window-based online processing method with the preprocessing method of iDetector, which directly aggregates channels. Detection latency time (DLT) is defined as the time difference between receiving the first packet of a sample and outputting the judgment result for that sample. Specifically, we monitor the actual traffic on the server's network interface over a period of time and calculate the average, maximum, and minimum DLT for both preprocessing methods. It is worth noting that in the iDetector paper, the authors used a default setting of $\omega = 224$. For a fair comparison, in addition to this default setting, we also conduct tests with $\omega = 30$ (consistent with our settings).

From the boxplot of DLT in Fig. 11, it is evident that our proposed time-window-based online processing method better fulfills real-time detection requirements. The DLT values exhibit a concentrated distribution, as our detection latency is primarily determined by the size of the time window ($t = 1$ s). Particularly, the majority of the DLT for the lightweight model falls between 1.07 and 1.11 s, with an average detection delay of 1.10 s. This indicates that the actual time spent on processing and prediction for multiple channel traffic within a time window is only 100 ms. Although the full RCML-IDS model shows slightly higher latency due to its increased complexity, these results collectively demonstrate that our time-window sampling mechanism provides a unified foundation for real-time detection across models of varying scales, showcasing the advantages of our preprocessing approach.

In contrast, iDetector's online processing method, which relies solely on the number of packets within a channel sample, results in significantly higher detection latency. Its default configuration shows particularly poor performance: maximum latency reaches 1851 s, average latency stands at 924 s, and even the minimum latency remains as high as 135 s. Reducing ω , that is, lowering the requirement for the number of packets per sample, can improve response speed to some extent. At $\omega = 30$, iDetector achieves a reduced average detection latency of 111 s, representing a significant improvement over the default configuration. Nevertheless, when compared with our time-window-based method, its detection latency remains excessively long.

TABLE VII

DETECTION PERFORMANCE AND COMPUTATIONAL COST OF RCML-LIGHT ON RASPBERRY PI

Model	ACC	TPR	F1	DPT(ms)	MIT(ms)
RCML-light	96.98	96.98	96.97	7.80	52.37

DPT: Data Processing Time (*ms/sample*);

MIT: Model Inference Time (*ms/sample*);

Therefore, our method enables the system to process and detect traffic at a more stable and rapid pace. In contrast, iDetector's latency fluctuates widely, indicating that its online processing strategy has limitations and is unsuitable for real-time detection scenarios in IoT with stringent latency requirements.

G. Testing on Resource-Constrained Device

This section evaluates the real-time performance of the proposed approach on resource-constrained devices to verify its practical applicability in IoT edge environments. The experiments are conducted on a Raspberry Pi 4B, equipped with an ARM Cortex-A72 quad-core processor (1.50 GHz, 1 MB L2 cache) and 4 GB of LPDDR4 RAM. This platform is widely adopted as a benchmark in edge AI research, with resource specifications falling within the typical range of common IoT nodes. Given the high computational complexity of the Transformer and LSTM modules used in our model, a lightweight configuration is adopted to fit limited resources. Specifically, based on the RCML-light model without the pretraining mechanism, the number of layers in both the Transformer and LSTM is reduced from 2 to 1. Tests show that an input sample length of 10 ($n = 10$) is the optimal feasible setting under the resource constraints of this device.

Table VII presents the sample construction delay (DPT), inference delay (MIT), and classification performance metrics under the aforementioned configuration. The results demonstrate that RCML-light attains millisecond-level real-time detection on the Raspberry Pi, a resource-constrained device while maintaining high classification accuracy and $F1$ score. This suggests that the lightweight model can be effectively deployed on IoT edge devices.

VI. CONCLUSION

This article significantly enhances the capability and real-time performance of encrypted traffic detection through a unique online traffic preprocessing mechanism, a multimodal learning framework, and a channel-level traffic aggregation approach. Just as humans can better understand video content by combining image and audio information, this article leverages DL models to fuse raw byte and packet length features as two modalities. In combination with channel-level traffic aggregation, the proposed approach enables a comprehensive analysis of the inherent complex patterns of encrypted traffic at a coarser granularity, thereby allowing for a more precise distinction of various malicious behaviors.

Through extensive experimental evaluations, we demonstrate the superior performance of RCML-IDS compared to state-of-the-art solutions. More importantly, the proposed

online preprocessing mechanism based on time-window sampling enables the model to capture packets in real-time by time windows (e.g., 1 s), and immediately input them into the model for detection and output of results. This mechanism allows the lightweight version, RCML-light, to attain an average detection latency of 1.10 s, with a total processing and inference time of approximately 20 ms per sample, meeting the real-time requirements of general IoT scenarios. Furthermore, deployment experiments on a Raspberry Pi 4B validate the feasibility and efficiency of the proposed method in resource-constrained edge devices.

Future work will focus on two main directions: first, investigating the model's robustness against adversarial traffic and evasion attacks to enhance its applicability in complex threat environments; second, advancing its integration with existing IDS to explore its potential for deployment in real-world IoT networks.

REFERENCES

- [1] J. Tong and Y. Zhang, "A real-time label-free self-supervised deep learning intrusion detection for handling new type and few-shot attacks in IoT networks," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 30769–30786, Oct. 2024, doi: [10.1109/JIOT.2024.3414492](https://doi.org/10.1109/JIOT.2024.3414492).
- [2] (2023). 2023 Enterprise IoT & OT Threat Report. [Online]. Available: <https://www.zayo.com/resources/2023-threatlabz-enterprise-iot-ot-threat-report>
- [3] IoT Business News. (2024). *AI Adoption and IoT Proliferation Fuel 82% Spike in DDoS Attacks in 2024* [EB/OL]. [Online]. Available: <https://iotbusinessnews.com/2025/02/21/26269-ai-adoption-and-iot-proliferation-fuel-82-spike-in-ddos-attacks-in-2024/>
- [4] T. Dierks and E. Rescorla. (2008). *The Transport Layer Security (TLS) Protocol Version 1.2*, 2008. [Online]. Available: <https://tools.ietf.org/html/rfc5246>
- [5] A. Freier, P. Karlton, and P. Kocher, "The secure sockets layer (SSL) protocol version 3.0," Internet Eng. Task Force (IETF), Fremont, CA, USA, Tech. Rep. RFC6101, Aug. 2011.
- [6] Z. Song et al., "I² RNN: An incremental and interpretable recurrent neural network for encrypted traffic classification," *IEEE Trans. Dependable Secure Comput.*, early access, Feb. 28, 2023, doi: [10.1109/TDSC.2023.3245411](https://doi.org/10.1109/TDSC.2023.3245411).
- [7] S. Shi, D. Han, and M. Cui, "A multimodal hybrid parallel network intrusion detection model," *Connection Sci.*, vol. 35, no. 1, Dec. 2023, Art. no. 2227780.
- [8] T. Kim, B. Kang, M. Rho, S. Sezer, and E. G. Im, "A multimodal deep learning method for Android malware detection using various features," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 3, pp. 773–788, Mar. 2019.
- [9] M. B. Shaikh, D. Chai, S. M. S. Islam, and N. Akhtar, "Multimodal fusion for audio-image and video action recognition," *Neural Comput. Appl.*, vol. 36, no. 10, pp. 5499–5513, Apr. 2024.
- [10] Y. Zhao, H. Zhao, X. Zhang, and W. Liu, "Vehicle classification based on audio-visual feature fusion with low-quality images and noise," *J. Intell. Fuzzy Syst.*, vol. 45, no. 5, pp. 8931–8944, Nov. 2023.
- [11] L. Cheng, R. Cheng, Y. Liu, D. Gong, Z. Wang, and S. Ji, "Lip-audio modality fusion for deep forgery video detection," *Comput., Mater. Continua*, vol. 82, no. 2, pp. 3499–3515, 2025.
- [12] S. Cui, C. Dong, M. Shen, Y. Liu, B. Jiang, and Z. Lu, "CBSeq: A channel-level behavior sequence for encrypted malware traffic detection," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5011–5025, 2023.
- [13] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, and J. Weng, "Seeing traffic paths: Encrypted traffic classification with path signature features," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2166–2181, 2022, doi: [10.1109/TIFS.2022.3179955](https://doi.org/10.1109/TIFS.2022.3179955).
- [14] R. Li, X. Xiao, S. Ni, H. Zheng, and S. Xia, "Byte segment neural network for network traffic classification," in *Proc. IEEE/ACM 26th Int. Symp. Quality Service (IWQoS)*, Jun. 2018, pp. 1–10.
- [15] J. Dai, X. Xu, H. Gao, X. Wang, and F. Xiao, "SHAPE: A simultaneous header and payload encoding model for encrypted traffic classification," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 1993–2012, Jun. 2023.
- [16] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smart-phone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 63–78, Jan. 2018.
- [17] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023.
- [18] M. Shen, Y. Liu, L. Zhu, X. Du, and J. Hu, "Fine-grained webpage fingerprinting using only packet length information of encrypted traffic," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2046–2059, 2021.
- [19] G. Apruzzese, M. Andreolini, M. Colajanni, and M. Marchetti, "Hardening random forest cyber detectors against adversarial attacks," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 4, no. 4, pp. 427–439, Aug. 2020, doi: [10.1109/TETCI.2019.2961157](https://doi.org/10.1109/TETCI.2019.2961157).
- [20] R. Bozkır, M. Cicioğlu, A. Çalhan, and C. Toğay, "A new platform for machine-learning-based network traffic classification," *Comput. Commun.*, vol. 208, pp. 1–14, Aug. 2023.
- [21] H. Mohanty, A. H. Roudsari, and A. H. Lashkari, "Robust stacking ensemble model for darknet traffic classification under adversarial settings," *Comput. Secur.*, vol. 120, Mar. 2022, Art. no. 102830.
- [22] F. Zaki, F. Afifi, S. Abd Razak, A. Gani, and N. B. Anuar, "GRAIN: Granular multi-label encrypted traffic classification using classifier chain," *Comput. Netw.*, vol. 213, Aug. 2022, Art. no. 109084.
- [23] N. Moustafa, B. Turnbull, and K.-K.-R. Choo, "An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4815–4830, Jun. 2019, doi: [10.1109/JIOT.2018.2871719](https://doi.org/10.1109/JIOT.2018.2871719).
- [24] C. Zhang, Z. Lian, H. Huang, and C. Su, "PCIDS: Permission and credibility-based intrusion detection system in IoT gateways," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 904–913, Jan. 2024, doi: [10.1109/JIOT.2023.3289206](https://doi.org/10.1109/JIOT.2023.3289206).
- [25] J. Huang et al., "A fine-grained video traffic control mechanism in software-defined networks," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 3, pp. 3501–3515, Sep. 2022.
- [26] S. Feghhi and D. J. Leith, "A web traffic analysis attack using only timing information," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 8, pp. 1747–1759, Aug. 2016.
- [27] Z. Zhao et al., "ERNN: Error-resilient RNN for encrypted traffic detection towards network-induced phenomena," *IEEE Trans. Dependable Secure Comput.*, early access, Feb. 3, 2023, doi: [10.1109/TDSC.2023.3242134](https://doi.org/10.1109/TDSC.2023.3242134).
- [28] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.
- [29] Y. Qing et al., "Low-quality training data only? A robust framework for detecting encrypted malicious network traffic," 2023, *arXiv:2309.04798*.
- [30] Y. Han, X. Wang, M. He, X. Wang, and S. Guo, "Intrusion detection for encrypted flows using single feature based on graph integration theory," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17589–17601, May 2024.
- [31] C. Liu, Z. Cao, G. Xiong, G. Gou, S.-M. Yiu, and L. He, "MaMPF: Encrypted traffic classification based on multi-attribute Markov probability fingerprints," in *Proc. IEEE/ACM 26th Int. Symp. Quality Service (IWQoS)*, Jun. 2018, pp. 1–10.
- [32] Y. Wang, H. He, Y. Lai, and A. X. Liu, "A two-phase approach to fast and accurate classification of encrypted traffic," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1071–1086, Jun. 2023.
- [33] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Inform. (ISI)*, Jul. 2017, pp. 43–48.
- [34] C. Dong, C. Zhang, Z. Lu, B. Liu, and B. Jiang, "CETAnalytics: Comprehensive effective traffic information analytics for encrypted traffic classification," *Comput. Netw.*, vol. 176, Jul. 2020, Art. no. 107258.
- [35] W. Maonan, Z. Kangfeng, X. Ning, Y. Yanqing, and W. Xiujuan, "CENTIME: A direct comprehensive traffic features extraction for encrypted traffic classification," in *Proc. IEEE 6th Int. Conf. Comput. Commun. Syst. (ICCCS)*, Apr. 2021, pp. 490–498.
- [36] X. Meng, Y. Wang, R. Ma, H. Luo, X. Li, and Y. Zhang, "Packet representation learning for traffic classification," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 3546–3554.
- [37] J. Gao, Y. Lu, Y. He, M. Fan, D. Han, and Y. Qiao, "Tokenization representation and deep-learning-based intrusion detection in Internet of Vehicles," *IEEE Internet Things J.*, vol. 11, no. 23, pp. 37974–37987, Dec. 2024, doi: [10.1109/JIOT.2024.3441763](https://doi.org/10.1109/JIOT.2024.3441763).
- [38] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, Feb. 2023, pp. 1–8.

- [39] X. Kong, Y. Zhou, Y. Xiao, X. Ye, H. Qi, and X. Liu, "IDetector: A novel real-time intrusion detection solution for IoT networks," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31153–31166, Oct. 2024.
- [40] H. He, X. Sun, H. He, G. Zhao, L. He, and J. Ren, "A novel multimodal-sequential approach based on multi-view features for network intrusion detection," *IEEE Access*, vol. 7, pp. 183207–183221, 2019, doi: [10.1109/ACCESS.2019.2959131](https://doi.org/10.1109/ACCESS.2019.2959131).
- [41] Y. Yu, L. Xu, and X. Jiang, "A high-performance multimodal deep learning model for detecting minority class sample attacks," *Symmetry*, vol. 16, no. 1, p. 42, Jan. 2023.
- [42] X. Yun, Y. Wang, Y. Zhang, C. Zhao, and Z. Zhao, "Encrypted TLS traffic classification on cloud platforms," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 164–177, Feb. 2023.
- [43] X. Tan et al., "Inter-flow spatio-temporal correlation analysis based website fingerprinting using graph neural network," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7619–7632, 2024.
- [44] X. Wang, S. Chen, and J. Su, "App-Net: A hybrid neural network for encrypted mobile traffic classification," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Sep. 2020, pp. 424–429.
- [45] A. Alrefaei and M. Ilyas, "Using machine learning multiclass classification technique to detect IoT attacks in real time," *Sensors*, vol. 24, no. 14, p. 4516, Jul. 2024.
- [46] J. Wang, H. Xu, Y. G. Achamyaleh, S. Huang, and M. A. A. Faruque, "HyperDetect: A real-time hyperdimensional solution for intrusion detection in IoT networks," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14844–14856, Apr. 2024.
- [47] H. Yan, X. Lin, S. Li, H. Peng, and B. Zhang, "Global or local adaptation? client-sampled federated meta-learning for personalized IoT intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 279–293, 2025, doi: [10.1109/TIFS.2024.3516548](https://doi.org/10.1109/TIFS.2024.3516548).
- [48] J. Devlin, M. W. Chang, and K. Lee, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, vol. 1, 2019, pp. 4171–4186.
- [49] H. Hindy, C. Tachtatzis, R. Atkinson, E. Bayne, and X. Bellekens, "MQTT-IoT-IDS2020: MQTT Internet of Things intrusion detection dataset," *IEEE Dataport*, Jun. 2020, doi: [10.21227/bhxy-ep04](https://doi.org/10.21227/bhxy-ep04).
- [50] T. M. Booij, I. Chiscop, E. Meeuwissen, N. Moustafa, and F. T. H. D. Hartog, "ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 485–496, Jan. 2022.
- [51] S. Dadkhah, E. C. P. Neto, R. Ferreira, R. C. Molokwu, S. Sadeghi, and A. A. Ghorbani, "CICIoMT2024: Attack vectors in healthcare devices—A multi-protocol dataset for assessing IoMT device security," *Internet Things*, vol. 28, pp. 1–30, Dec. 2024.



Xiaowei Zhao is currently pursuing the Ph.D. degree in electronic science and technology with Beijing University of Posts and Telecommunications, Beijing, China.

Her research interests include machine learning and network security.



Mingshu He (Member, IEEE) received the Ph.D. degree from Beijing University of Posts and Telecommunications, Beijing, China, in 2022.

He is currently doing research at the School of Cyberspace Security, Beijing University of Posts and Telecommunications. His research interests include network security, anomaly detection, and machine learning.



Xiaojuan Wang (Member, IEEE) received the Ph.D. degree in electronic science and technology from Beijing University of Posts and Telecommunications, Beijing, China, in 2015.

She is currently an Associate Professor with the School of Electronic Engineering, Beijing University of Posts and Telecommunications. Her research interests include deep learning, complex networks, and human gesture recognition.